

Plan de Pruebas y Calidad de Software

Entrega 1: Plan de Pruebas Inicial, Análisis Estático y Suite de Pruebas Dockerizada

Asignatura:	Calidad y Pruebas de Software
Estudiantes:	Santiago Aguirre Salinas & Manuel Felipe Castellanos Calderón
Universidad:	Universidad de Caldas
Idea de negocio:	Registro y Control de Mantenimiento de Vehículos

Sección I: Idea de Negocio y Aplicación Generada

1. Descripción de la idea

La aplicación **Bitácora de Mantenimiento Vehicular** permite registrar y controlar el mantenimiento de una flota de vehículos. Por cada vehículo se registra su placa, el kilometraje actual, el kilometraje en el que corresponde el próximo mantenimiento, la categoría del servicio (aceite, frenos, llantas, filtros u otro), la fecha y notas opcionales. El sistema calcula automáticamente cuántos kilómetros faltan para el próximo servicio y clasifica cada vehículo como *Al día*, *Próximo* o *Alerta Urgente*.

Reglas de negocio implementadas:

- El kilometraje próximo de mantenimiento debe ser mayor al kilometraje actual.
- Si faltan menos de 500 km para el servicio, la aplicación activa el estado "Alerta Urgente".
- No se permite guardar un mantenimiento sin seleccionar la categoría del servicio.

2. Prompt utilizado

Se utilizó el prompt provisto en la guía de la actividad, actuando la IA como Desarrollador Software Junior, con la idea de negocio de mantenimiento vehicular insertada en la sección correspondiente, solicitando una aplicación de un solo archivo HTML (HTML + CSS + JavaScript nativo), persistencia simulada con `localStorage`, y entre 5 y 10 defectos de calidad deliberados no revelados por la IA, detectables solo mediante pruebas adecuadas (checklists, análisis estático y pruebas unitarias).

El código completo generado se entrega en `src/index.html` (interfaz) y `src/logic.js` (lógica de negocio, separada para poder probarla con Jest).

Sección II: Listas de Chequeo (Checklists)

1. Lista de Chequeo de Requerimientos y Funcionalidad

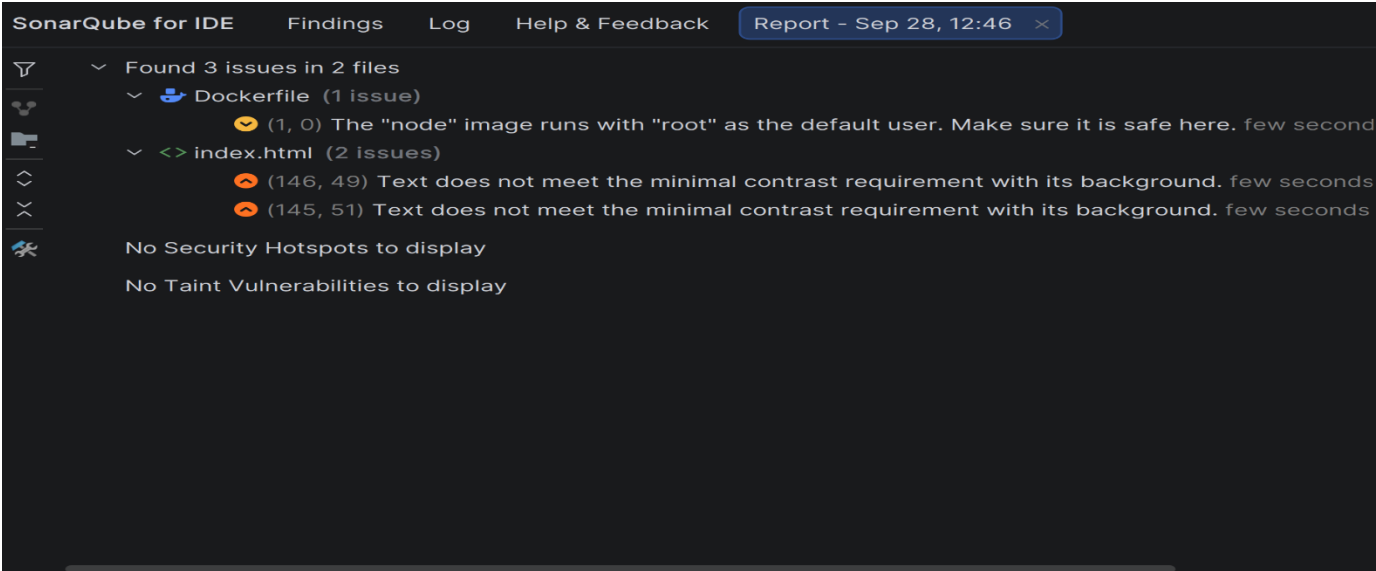
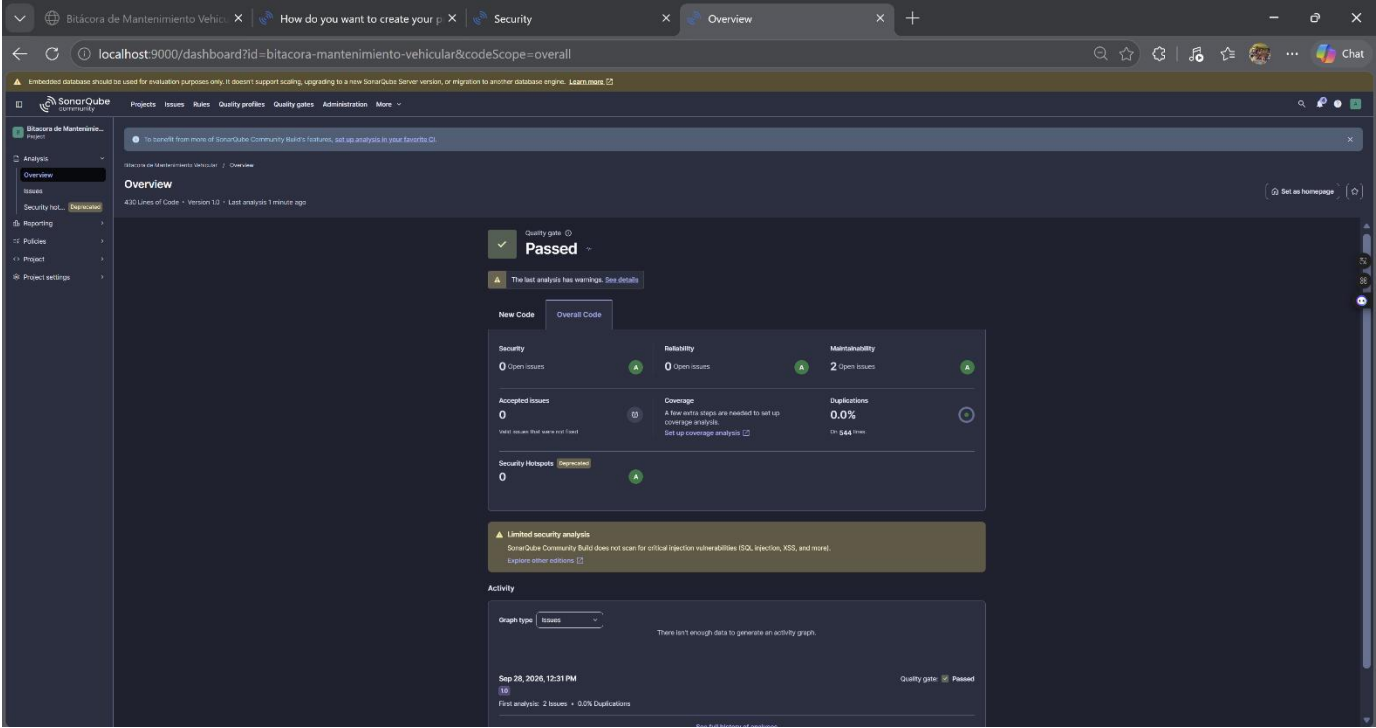
ID	Requerimiento / Criterio de Aceptación	Estado	Observaciones / Defecto Encontrado
REQ01	El kilometraje próximo debe ser mayor al kilometraje actual.	Pasa	Funciona según lo esperado; rechaza valores iguales o menores.
REQ02	No se debe permitir guardar un mantenimiento sin categoría de servicio.	Falla	Solo se valida al crear un registro nuevo; al editar un registro se puede dejar la categoría vacía y se guarda igual.
REQ03	Si faltan menos de 500 km, el estado debe ser "Alerta Urgente".	Falla	A exactamente 500 km restantes ya se marca como urgente (debería ser "Próximo"); además el contador del panel superior usa un criterio distinto (<500) al de la insignia por fila (≤500), por lo que ambos pueden no coincidir.
REQ04	El sistema no debe aceptar kilometrajes negativos.	Falla	Se puede registrar un vehículo con kilometraje actual negativo (ej. -100) sin ningún aviso.
REQ05	Un registro eliminado no debe reaparecer al recargar la página.	Falla	Al eliminar un registro se actualiza la tabla en pantalla, pero el cambio no se guarda en <code>localStorage</code> ; al recargar la página el registro eliminado vuelve a aparecer.
REQ06	Los campos numéricos deben rechazar texto no numérico.	Falla	Si se escribe texto no numérico en kilometraje, el valor se convierte en <code>NaN</code> silenciosamente; el registro se guarda mostrando "NaN km" y su estado de alerta queda incorrecto.
REQ07	Los caracteres especiales en placa/notas no deben romper la vista.	Falla	El texto ingresado se inserta con <code>innerHTML</code> sin escapar; caracteres como <code><</code> o <code>&</code> pueden romper el renderizado de la fila.

2. Lista de Chequeo de Calidad de Código (Code Review Manual)

Criterio	Cumple	Observación
Legibilidad: nombres de variables y funciones claros.	Sí	Nombres en español, coherentes con el dominio (<code>kmActual</code> , <code>kmProximo</code> , <code>estadoDe</code> , etc.).
Estructura: separación de responsabilidades (lógica vs. UI).	Parcial	La lógica de negocio se extrajo a <code>logic.js</code> , pero la manipulación del DOM y el manejo de <code>localStorage</code> siguen mezclados en el mismo script de <code>index.html</code> .
Manejo de errores: validaciones antes de operar sobre <code>localStorage</code> .	No	<code>loadRecords()</code> hace <code>JSON.parse</code> directo sobre el valor guardado, sin <code>try/catch</code> ; un valor corrupto en <code>localStorage</code> rompería la carga de toda la app.
Seguridad básica: sin inyección de scripts (<code>innerHTML</code> desprotegido).	No	Los campos de placa y categoría se insertan con <code>innerHTML</code> sin sanitizar la entrada del usuario.

Sección III: Pruebas Estáticas con SonarQube (Dockerizado)

1. Captura de pantalla del Dashboard de SonarQube



2. Defectos estáticos identificados (revisión manual del código)

Tipo	Severidad	Descripción del Hallazgo	Solución Sugerida
Vulnerability	High	Inyección potencial mediante <code>element.innerHTML</code> con datos de placa/categoría sin sanitizar.	Sustituir por <code>textContent</code> o escapar el HTML antes de insertar.
Bug	Critical	La operación de eliminar un registro no llama a <code>persist()</code> : el borrado no se guarda en <code>localStorage</code> .	Invocar <code>persist(records)</code> inmediatamente después de filtrar el arreglo.
Code Smell	Major	<code>parseInt(value)</code> usado sin especificar la base (<code>radix</code>), regla estándar de calidad JS.	Usar <code>parseInt(value, 10)</code> y validar <code>isNaN()</code> .
Code Smell	Major	Lógica de negocio duplicada e inconsistente: el umbral de "urgente" se calcula distinto en <code>estadoDe()</code> (≤ 500) y en <code>contarUrgentes()</code> (< 500).	Centralizar el umbral en una única constante y función reutilizada en ambos lugares.
Bug	Minor	<code>JSON.parse</code> sobre el valor de <code>localStorage</code> sin bloque <code>try/catch</code> .	Envolver la deserialización en <code>try/catch</code> y devolver un arreglo vacío ante datos corruptos.

Sección IV: Pruebas Unitarias (Dockerizadas)

Se implementaron 8 pruebas automatizadas con Jest sobre las funciones clave de `src/logic.js` (cálculo de kilometraje restante, clasificación de estado, validación de reglas de negocio y parseo de kilometraje).

1. Matriz de Casos de Prueba Unitarios

ID	Función a Testear	Datos de Entrada	Resultado Esperado	Resultado Obtenido
UT-01	kmRestante()	Actual: 45000, Próximo: 45100	100	100
UT-02	estadoDe()	Actual: 49500, Próximo: 50000 (500 restantes)	"warn"	"urgent" (<i>Bug detectado</i>)
UT-03	esKmValido()	Actual: 50000, Próximo: 50000	false	false
UT-04	estadoDe() vs contarUrgentes()	1 registro con 500 km restantes	Ambas coinciden	No coinciden (<i>Bug detectado</i>)
UT-05	validarRegistro()	Editando, categoría: ""	valid: false	valid: true (<i>Bug detectado</i>)
UT-06	parseKm()	"abc"	No debería ser NaN	NaN (<i>Bug detectado</i>)
UT-07	esKmValido()	Actual: -100, Próximo: 500	false	true (<i>Bug detectado</i>)
UT-08	validarRegistro()	Actual: 40000, Próximo: 45000, categoría: "aceite"	valid: true	valid: true

De 8 pruebas, 5 fallan contra la implementación actual (UT-02, UT-04, UT-05, UT-06, UT-07), confirmando 5 de los defectos deliberados introducidos en `logic.js`. El defecto de persistencia al eliminar (REQ05) y el de `innerHTML` sin sanitizar (REQ07) no se cubren aquí por ser de integración con el DOM/localStorage; quedan documentados en las listas de chequeo (Sección II) y en el hallazgo estático (Sección III).

2. Evidencia de ejecución

```
PS C:\Users\sanas\OneDrive\Escritorio\P & C\New folder\project> docker-compose run --rm unit-tests
time="2026-09-28T12:42:39-05:00" level=warning msg="C:\\Users\\sanas\\OneDrive\\Escritorio\\P & C\\New folder\\project\\docker-compose.yml: the attribute 'version' is obsolete, it
will be ignored, please remove it to avoid potential confusion"
Container project-unit-tests-run-beb51a158f57 Creating
Container project-unit-tests-run-beb51a158f57 Created

> bitacora-mantenimiento-vehicular@1.0.0 test
> jest --colors --verbose

FAIL tests/logic.test.js
  ✓ UT-01 kmRestante calcula la diferencia entre km próximo y km actual (9 ms)
  ✗ UT-02 estadoDe() a exactamente 500 km restantes NO debería ser urgente (9 ms)
  ✗ UT-03 esKmValido() rechaza km próximo igual al km actual (2 ms)
  ✗ UT-04 estadoDe() y contarUrgentes() deben coincidir en el límite de 500 km (22 ms)
  ✗ UT-05 validarRegistro() exige categoría también en modo edición (5 ms)
  ✗ UT-06 parseKm() no debería devolver NaN para una entrada no numérica (2 ms)
  ✗ UT-07 esKmValido() rechaza kilometraje actual negativo (2 ms)
  ✓ UT-08 validarRegistro() acepta un registro correcto (1 ms)

  ● UT-02 estadoDe() a exactamente 500 km restantes NO debería ser urgente

    expect(received).toBe(expected) // Object.is equality

    Expected: "warn"
    Received: "urgent"

      23 |   const esperado = 'warn'; // 500 km restantes: aún no es "menos de 500"
      24 |   const obtenido = estadoDe(49500, 50000);
    > 25 |   expect(obtenido).toBe(esperado); // Falla con la implementación actual -> Bug detectado
         |                               ^
      26 |   });
      27 |
      28 |   /**

    at Object.toBe (tests/logic.test.js:25:20)

  ● UT-04 estadoDe() y contarUrgentes() deben coincidir en el límite de 500 km

    expect(received).toBe(expected) // Object.is equality

    Expected: false
    Received: true

      41 |   const marcadoUrgentePorEstado = estadoDe(49500, 50000) === 'urgent';
      42 |   const marcadoUrgentePorContador = contarUrgentes(registros) === 1;
    > 43 |   expect(marcadoUrgentePorEstado).toBe(marcadoUrgentePorContador); // Falla -> Bug detectado (lógica duplicada e inconsistente)
         |                                   ^
      44 |   });
      45 |
      46 |   /**

    at Object.toBe (tests/logic.test.js:43:35)

  ● UT-05 validarRegistro() exige categoría también en modo edición

    expect(received).toBe(expected) // Object.is equality

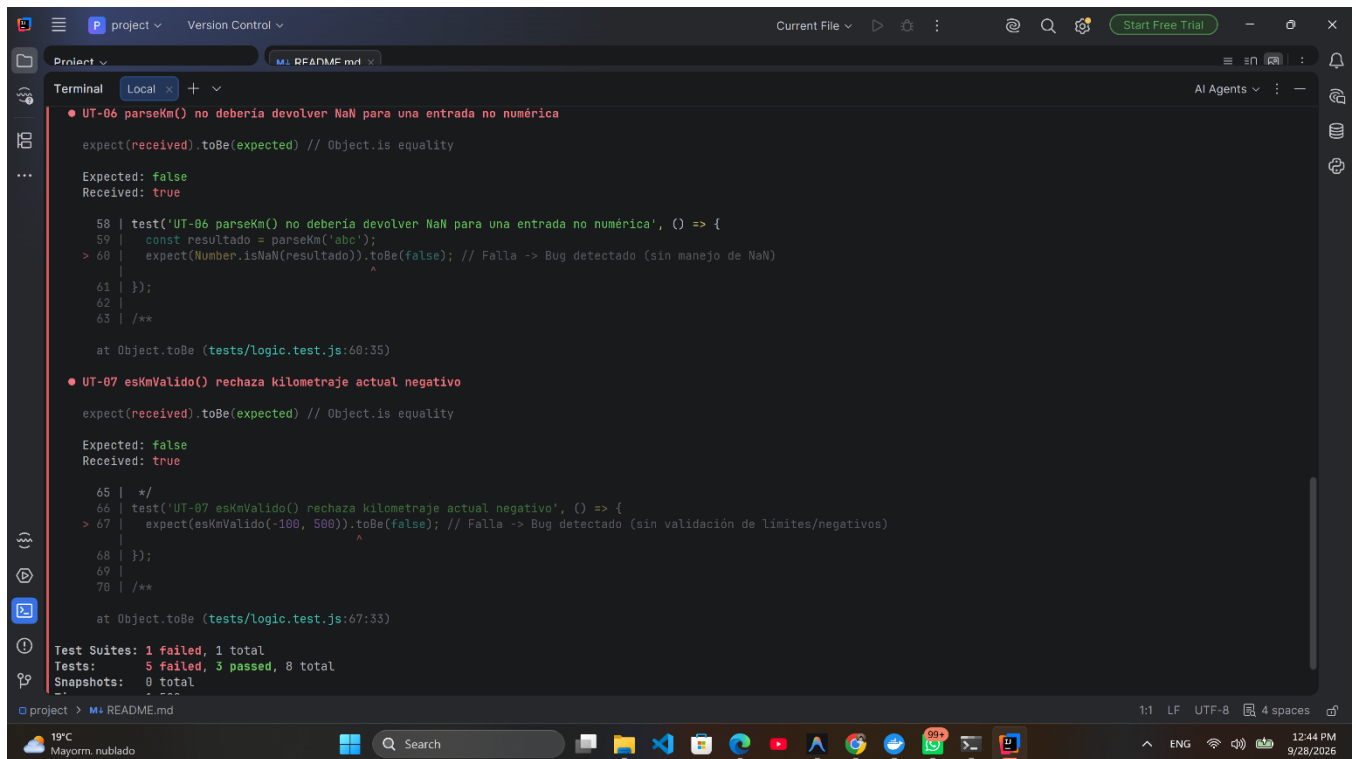
    Expected: false
    Received: true

      50 |   const registro = { kmActual: 1000, kmProximo: 2000, categoria: '' };
      51 |   const resultado = validarRegistro(registro, true); // isEditing = true
    > 52 |   expect(resultado.valid).toBe(false); // Falla con la implementación actual -> Bug detectado
         |                             ^
      53 |   });
      54 |
      55 |   /**

    at Object.toBe (tests/logic.test.js:52:27)

  ● UT-06 parseKm() no debería devolver NaN para una entrada no numérica

    expect(received).toBe(expected) // Object.is equality
```



```
● UT-06 parseKm() no deberia devolver NaN para una entrada no numérica

expect(received).toBe(expected) // Object.is equality

Expected: false
Received: true

58 | test('UT-06 parseKm() no deberia devolver NaN para una entrada no numérica', () => {
59 |   const resultado = parseKm('abc');
> 60 |   expect(Number.isNaN(resultado)).toBe(false); // Falla -> Bug detectado (sin manejo de NaN)
    |                                     ^
61 | });
62 |
63 | /**

at Object.toBe (tests/logic.test.js:60:35)

● UT-07 esKmValido() rechaza kilometraje actual negativo

expect(received).toBe(expected) // Object.is equality

Expected: false
Received: true

65 | /*
66 | test('UT-07 esKmValido() rechaza kilometraje actual negativo', () => {
> 67 |   expect(esKmValido(-100, 500)).toBe(false); // Falla -> Bug detectado (sin validación de límites/negativos)
    |                                     ^
68 | });
69 |
70 | /**

at Object.toBe (tests/logic.test.js:67:33)

Test Suites: 1 failed, 1 total
Tests:       5 failed, 3 passed, 8 total
Snapshots:  0 total
```

3. Entorno Docker

El archivo `docker-compose.yml` del repositorio define dos servicios: `sonarqube` para el análisis estático (Sección III) y `unit-tests`, construido a partir del `Dockerfile` incluido, que instala las dependencias con `npm` y ejecuta `npm test` dentro del contenedor.